LAPORAN

WORKSHOP SISTEM INFORMASI WEB SERVER

(TIF330805)

MINGGU 9 PERTEMUAN 1

SEMESTER III

BKPM ACARA 17

Nama:
SEPTIAN DWI RAMADHAN PUTRA

NIM:
E41250282

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Pada Acara 17, setiap Controller membuat response JSON sendiri dengan http_response_code() dan json_encode(). Cara ini membuat kode berulang dan format response bisa berbeda beda antar endpoint. Padahal aplikasi yang memakai API membutuhkan format yang konsisten dan kode status HTTP yang tepat agar mudah memahami hasil sebuah request.

Pada praktikum ini dibuat class Response yang menjadi satu-satunya tempat pembentukan response JSON. Controller dan Router cukup memanggil Response::json(). Selain itu, kode status HTTP dipakai secara konsisten, termasuk 422 untuk data yang formatnya tidak valid.

2. Tujuan

1. Membuat class Response untuk membentuk response JSON yang seragam.
2. Merapikan Controller dan Router agar memakai Response::json().
3. Menerapkan HTTP status code yang sesuai, termasuk 422 untuk validasi data.
4. Menguji ulang seluruh endpoint, termasuk skenario gagal.

3. Dasar Teori Singkat

JSON encoding. json_encode() mengubah array PHP menjadi teks JSON. Flag tambahan yang dipakai: JSON_PRETTY_PRINT (hasil rapi dan mudah dibaca), JSON_UNESCAPED_SLASHES (tanda / tidak diubah), dan JSON_UNESCAPED_UNICODE (huruf non-ASCII tampil apa adanya).

HTTP status code. Kode angka dari server yang menunjukkan hasil request. Kode dikelompokkan menjadi 1xx (informasi), 2xx (berhasil), 3xx (pengalihan), 4xx (kesalahan client), dan 5xx (kesalahan server). Di PHP, kode dikirim dengan http_response_code().

Kode	Nama	Contoh penggunaan
200	OK	GET, PUT, DELETE berhasil
201	Created	Data baru berhasil dibuat (POST)
400	Bad Request	Format request tidak valid / data belum lengkap
404	Not Found	Data atau endpoint tidak ditemukan
405	Method Not Allowed	Method tidak diperbolehkan
409	Conflict	Konflik data, misalnya NIM sudah dipakai
422	Unprocessable Entity	Data lengkap tetapi gagal validasi
500	Internal Server Error	Kegagalan tak terduga di server

Response envelope. Format response yang selalu sama, berisi success (berhasil atau tidak), message (pesan), data (hasil), dan timestamp (waktu). Dengan format yang sama, client tidak perlu menangani banyak bentuk response yang berbeda.

4. Alat dan Bahan

- XAMPP (Apache dengan mod_rewrite dan MySQL)
- Visual Studio Code
- Postman
- Web Browser
- Database acara_7

HASIL PRAKTIK

1. Persiapan

Praktikum ini melanjutkan proyek sistem-informasi-akademik dari Acara 17 dengan database acara_7. Perubahan hanya pada lapisan Core (class Response baru), Controller, dan Router. Struktur Model tidak berubah, kecuali exception pada Model kini membawa kode HTTP (409 atau 422). File konfigurasi app/config/app.php ditambahkan untuk pengaturan APP_ENV dan panjang NIM.

Modul meminta NIM berjumlah 9 digit, tetapi data pada database acara_7 memakai NIM 7 digit (misalnya 2401001). Agar data yang sudah ada tetap valid, aturan pada praktikum ini adalah NIM 7 digit, diatur lewat konstanta NIM_LENGTH.

2. Class Response

Class Response (app/core/Response.php) memiliki satu method static, json(). Method ini mengirim kode status, header JSON, lalu isi response, dan menghentikan program (exit). Isi response selalu berisi success, message, data, dan timestamp. Field errors hanya muncul bila ada daftar kesalahan (Tugas Mandiri 1), dan pretty print hanya aktif pada mode development (Tugas Mandiri 2). Karena berada di folder core, class ini otomatis ditemukan autoloader.

```
// Response: satu-satunya tempat pembentukan response JSON, sehingga seluruh
// response API (dari Router maupun Controller) memiliki format yang sama:
//   { "success": ..., "message": ..., "data": ..., "errors": {...}, "timestamp": ...
}
// Field "errors" hanya muncul jika ada daftar kesalahan validasi per field.
class Response {
    public static function json(int $httpCode, bool $success, string $message, $data
= null, ?array $errors = null): void {
        http_response_code($httpCode);
        header('Content-Type: application/json');

        $body = [
            'success' => $success,
            'message' => $message,
            'data'    => $data,
        ];

        if ($errors !== null) {
            $body['errors'] = $errors;
        }

        $body['timestamp'] = date('c');

        $flags = JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE;

        // JSON_PRETTY_PRINT hanya pada mode pengembangan.
        if (defined('APP_ENV') && APP_ENV === 'development') {
            $flags |= JSON_PRETTY_PRINT;
        }
```

```
        echo json_encode($body, $flags);
        exit;
    }
}
```

3. Refactor Controller

Seluruh http_response_code() dan echo json_encode() pada Controller diganti dengan Response::json(). Perbandingan method index() pada MahasiswaController:

Sebelum (Acara 17):

```
    // GET /mahasiswa  atau  GET /mahasiswa?id=1
    public function index(): void {
        header('Content-Type: application/json');

        $id = $_GET['id'] ?? null;

        if ($id !== null) {
            $data = ctype_digit((string) $id) ? $this->model->find((int) $id) :
false;

            if (!$data) {
                http_response_code(404);
                echo json_encode(['success' => false, 'message' => 'Data tidak
ditemukan']);
                exit;
            }

            echo json_encode(['success' => true, 'data' => $data]);
            exit;
        }

        echo json_encode(['success' => true, 'data' => $this->model->getAll()]);
    }

    // POST /mahasiswa
    public function store(): void {
        header('Content-Type: application/json');

        $data  = json_decode(file_get_contents('php://input'), true);
        $error = $this->validasi($data);

        if ($error) {
            http_response_code(400);
            echo json_encode(['success' => false, 'message' => $error]);
            exit;
        }

        $data['status'] = $data['status'] ?? 'aktif';

        try {
            $id = $this->model->create($data);
        } catch (InvalidArgumentException $e) {
            http_response_code(400);
            echo json_encode(['success' => false, 'message' => $e->getMessage()]);
            exit;
        }
```

```
        http_response_code(201);
        echo json_encode([
            'success' => true,
            'message' => 'Data berhasil ditambahkan',
            'data'    => ['id' => $id],
        ]);
    }
```

Sesudah (Acara 18):

```
    // GET /mahasiswa  atau  GET /mahasiswa?id=1
    public function index(): void {
        $id = $_GET['id'] ?? null;

        if ($id !== null) {
            Response::json(200, true, 'Data berhasil diambil', $this-
>ambilData($id));
        }

        Response::json(200, true, 'Data berhasil diambil', $this->model->getAll());
    }

    // POST /mahasiswa
    public function store(): void {
        $data = json_decode(file_get_contents('php://input'), true);
        $this->validasi($data);

        $data['status'] = $data['status'] ?? 'aktif';

        try {
            $id = $this->model->create($data);
        } catch (InvalidArgumentException $e) {
            Response::json($e->getCode() ?: 422, false, $e->getMessage());
        }

        Response::json(201, true, 'Data berhasil ditambahkan', ['id' => $id]);
    }
```

Controller menjadi lebih ringkas karena pembentukan JSON sudah menjadi tugas Response. Pengecekan ID dipindahkan ke method kecil ambilData() agar tidak ditulis berulang pada index(), update(), dan destroy(). ID yang bukan angka dijawab 400 dan data yang tidak ada dijawab 404.

```
    // Mengambil satu data. ID bukan angka -> 400, data tidak ada -> 404.
    private function ambilData($id): array {
        if (!ctype_digit((string) $id)) {
            Response::json(400, false, 'ID harus berupa angka');
        }

        $data = $this->model->find((int) $id);
        if (!$data) {
            Response::json(404, false, 'Data tidak ditemukan');
        }
```

```
        return $data;
    }
```

4. Validasi Data dengan Kode 400 dan 422

Validasi dibagi dua tahap. Tahap pertama memeriksa kelengkapan: body harus JSON dan field wajib terisi, bila tidak dijawab 400. Tahap kedua memeriksa format: NIM harus 7 digit angka, prodi_id dan angkatan harus angka, dan status harus aktif, cuti, atau lulus, bila tidak dijawab 422. Daftar kesalahan tiap field dikirim pada field errors. Sedangkan NIM yang sudah dipakai dijawab 409 dan program studi yang tidak ada dijawab 422 oleh Model.

```
    // Validasi data request.
    //  400 = data tidak lengkap (field wajib kosong / body bukan JSON)
    //  422 = data lengkap tetapi formatnya tidak valid (mis. NIM bukan 7 digit)
    // Daftar kesalahan per field dikirim pada field "errors".
    private function validasi($data): void {
        if (!is_array($data)) {
            Response::json(400, false, 'Body harus berupa JSON yang valid');
        }

        $errors = [];
        foreach (['nim', 'nama', 'prodi_id', 'angkatan'] as $field) {
            if (!isset($data[$field]) || !is_scalar($data[$field]) || trim((string)
$data[$field]) === '') {
                $errors[$field] = "Field $field wajib diisi";
            }
        }
        if ($errors) {
            Response::json(400, false, 'Data belum lengkap', null, $errors);
        }

        if (!ctype_digit((string) $data['nim']) || strlen((string) $data['nim']) !==
NIM_LENGTH) {
            $errors['nim'] = 'Format NIM tidak valid, harus ' . NIM_LENGTH . ' digit
angka';
        }
        if (!ctype_digit((string) $data['prodi_id'])) {
            $errors['prodi_id'] = 'prodi_id harus berupa angka';
        }
        if (!ctype_digit((string) $data['angkatan'])) {
            $errors['angkatan'] = 'angkatan harus berupa angka';
        }
        if (isset($data['status']) && !in_array($data['status'], ['aktif', 'cuti',
'lulus'], true)) {
            $errors['status'] = 'Status harus aktif, cuti, atau lulus';
        }
        if ($errors) {
            Response::json(422, false, 'Validasi gagal', null, $errors);
        }
    }
}
```

5. Merapikan Response pada Router

Blok 404 (endpoint tidak ditemukan), 405 (method tidak diizinkan), dan 500 (kesalahan database) pada Router juga memakai Response::json(). Begitu pula pesan koneksi database gagal pada Database.php. Dengan begitu seluruh response API memiliki format yang sama.

```php
// Router: membaca URL dan method HTTP, lalu meneruskannya ke Controller.
// Seluruh response (404, 405, 500) dibuat melalui Response::json().
class Router {
    // Peta sederhana: nama resource pada URL => nama class Controller
    private array $map = [
        'mahasiswa' => 'MahasiswaController',
        'prodi'     => 'ProdiController',
    ];

    public function dispatch(): void {
        $uri      = trim(parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH), '/');
        $segments = explode('/', $uri);
        $resource = end($segments) ?: 'mahasiswa';

        $controllerName = $this->map[$resource] ?? null;

        if (!$controllerName) {
            Response::json(404, false, 'Endpoint tidak ditemukan');
        }

        $controller = new $controllerName();

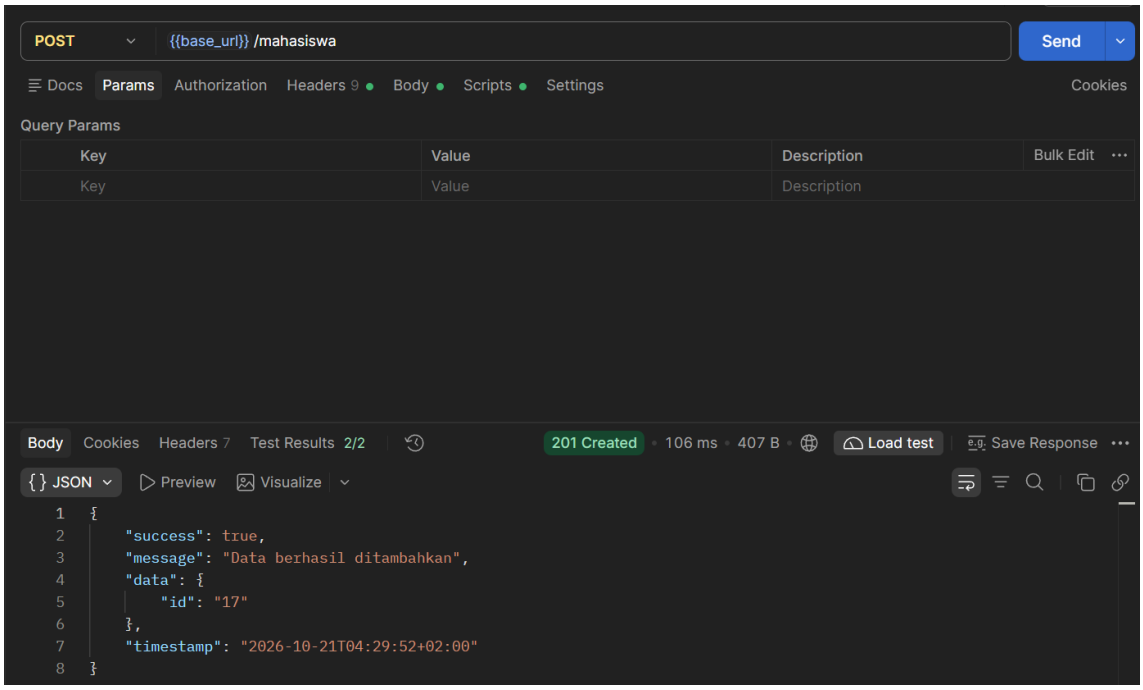
        try {
            switch ($_SERVER['REQUEST_METHOD']) {
                case 'GET':    $controller->index(); break;
                case 'POST':   $controller->store(); break;
                case 'PUT':    $controller->update($_GET['id'] ?? null); break;
                case 'DELETE': $controller->destroy($_GET['id'] ?? null); break;
                default:
                    Response::json(405, false, 'Method tidak diizinkan');
            }
        } catch (PDOException $e) {
            // Detail error database hanya dicatat, tidak dikirim ke client.
            error_log($e->getMessage());
            Response::json(500, false, 'Terjadi kesalahan pada server');
        }
    }
}
```

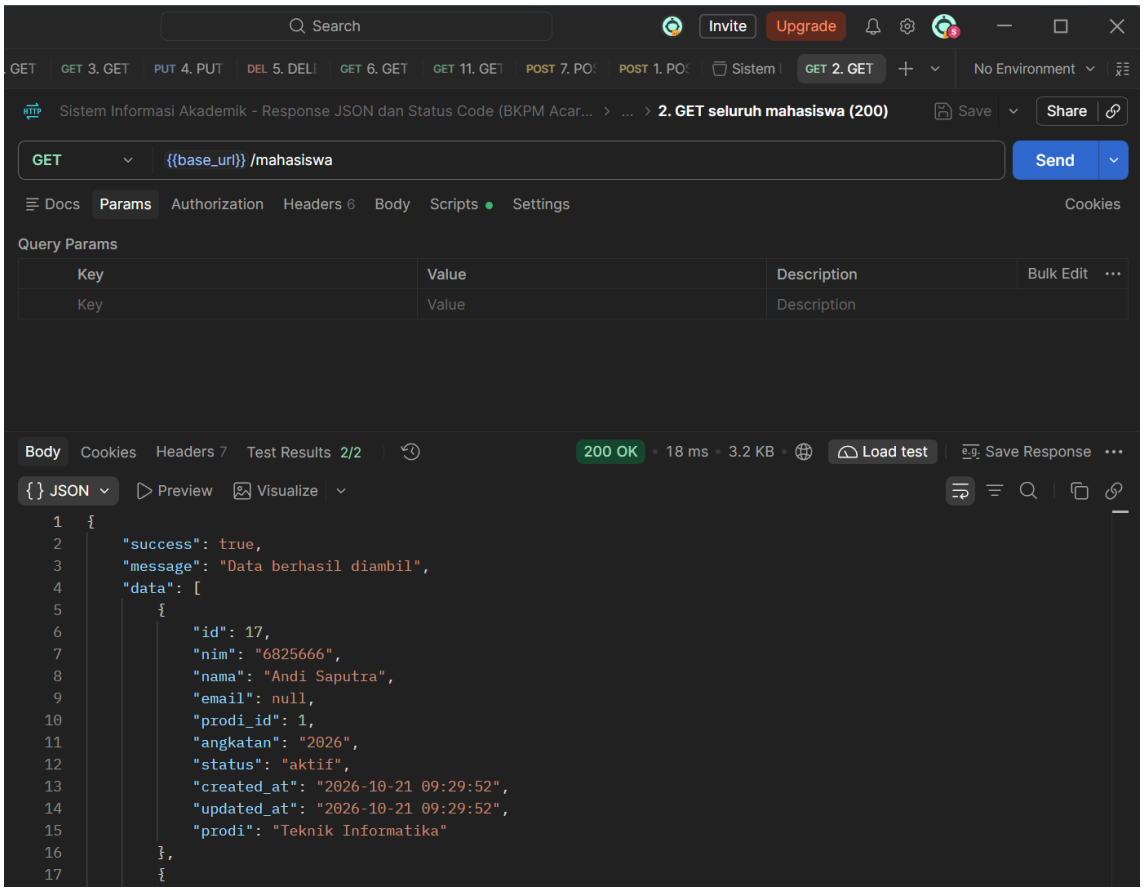
6. Pengujian Ulang

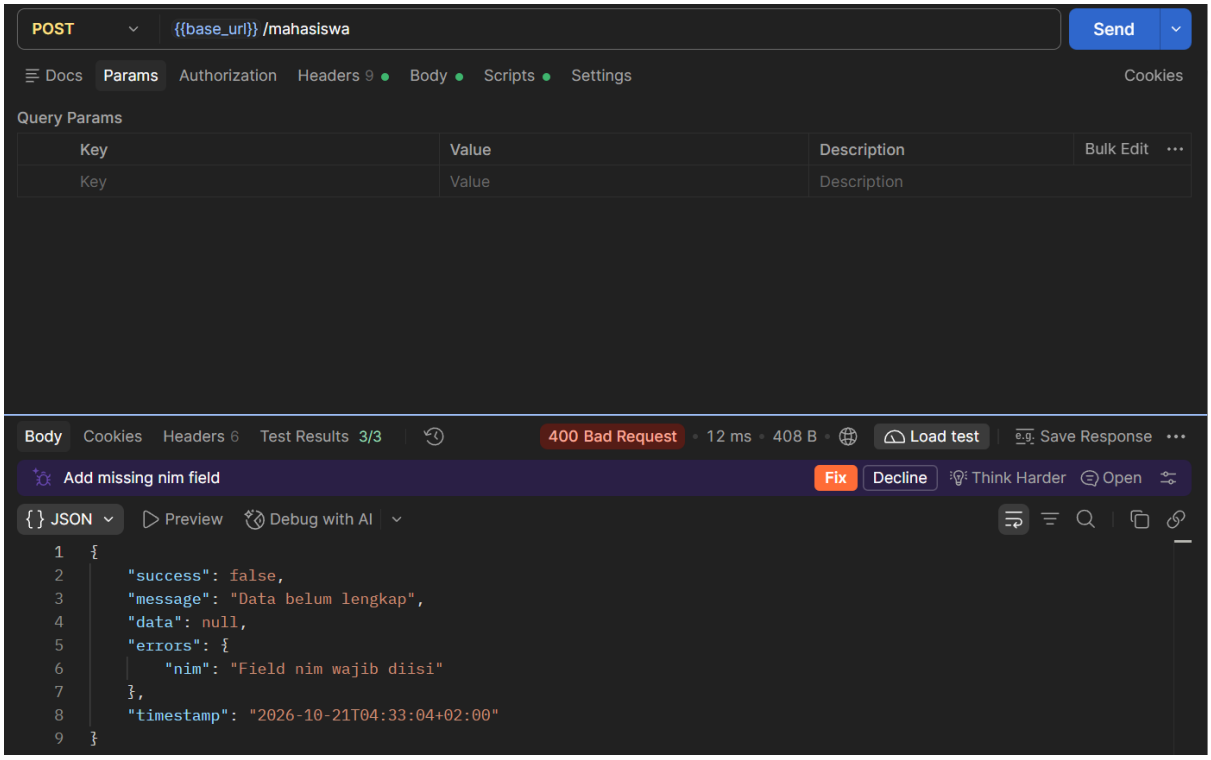
Seluruh endpoint diuji ulang dengan Postman, termasuk skenario gagal. Setiap response harus memiliki format yang sama dan kode status yang sesuai tabel.

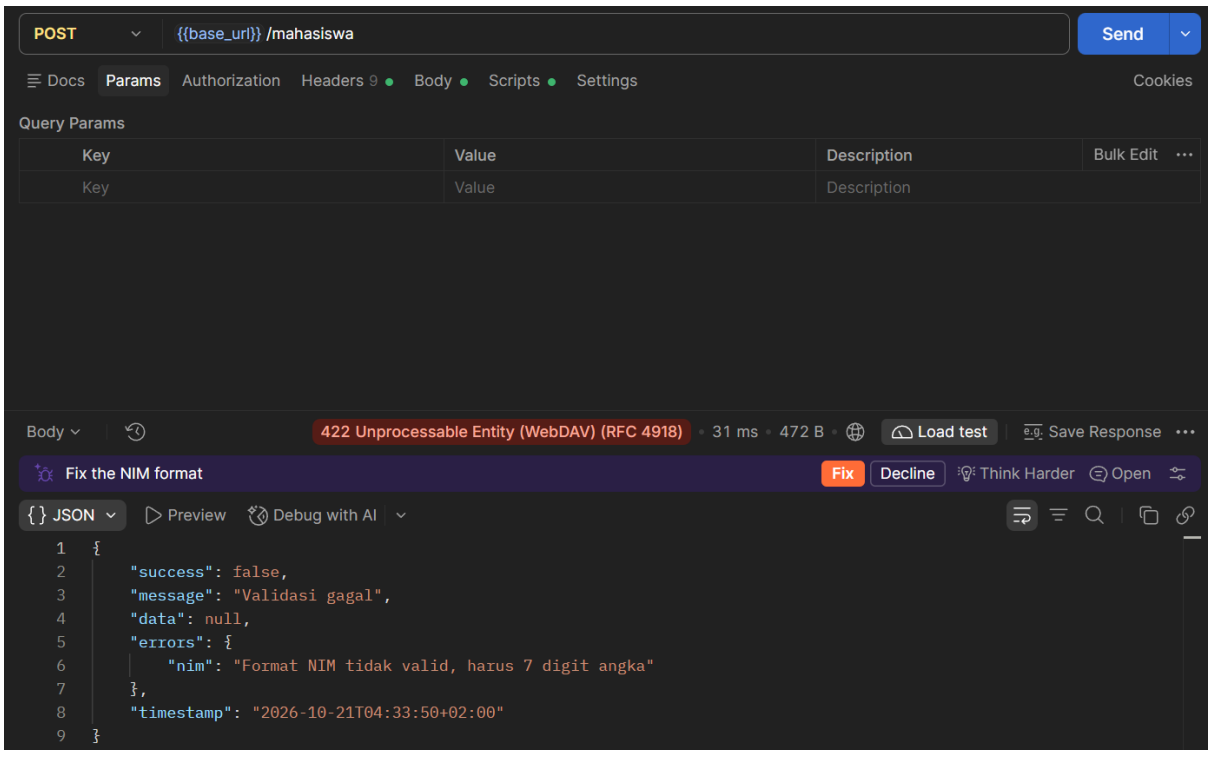
No	Request	Status	Hasil yang diharapkan
1	POST /mahasiswa (data valid)	201	Data dibuat, response berisi id
2	GET /mahasiswa	200	Seluruh mahasiswa
3	GET /mahasiswa?id={id}	200	Satu mahasiswa
4	PUT /mahasiswa?id={id}	200	Data diperbarui
5	DELETE /mahasiswa?id={id}	200	Data dihapus
6	GET /mahasiswa?id=9999	404	Data tidak ditemukan
7	POST NIM kosong	400	Data belum lengkap + errors.nim
8	POST format NIM salah (abc)	422	Validasi gagal + errors.nim
9	POST NIM sudah terdaftar	409	NIM sudah terdaftar

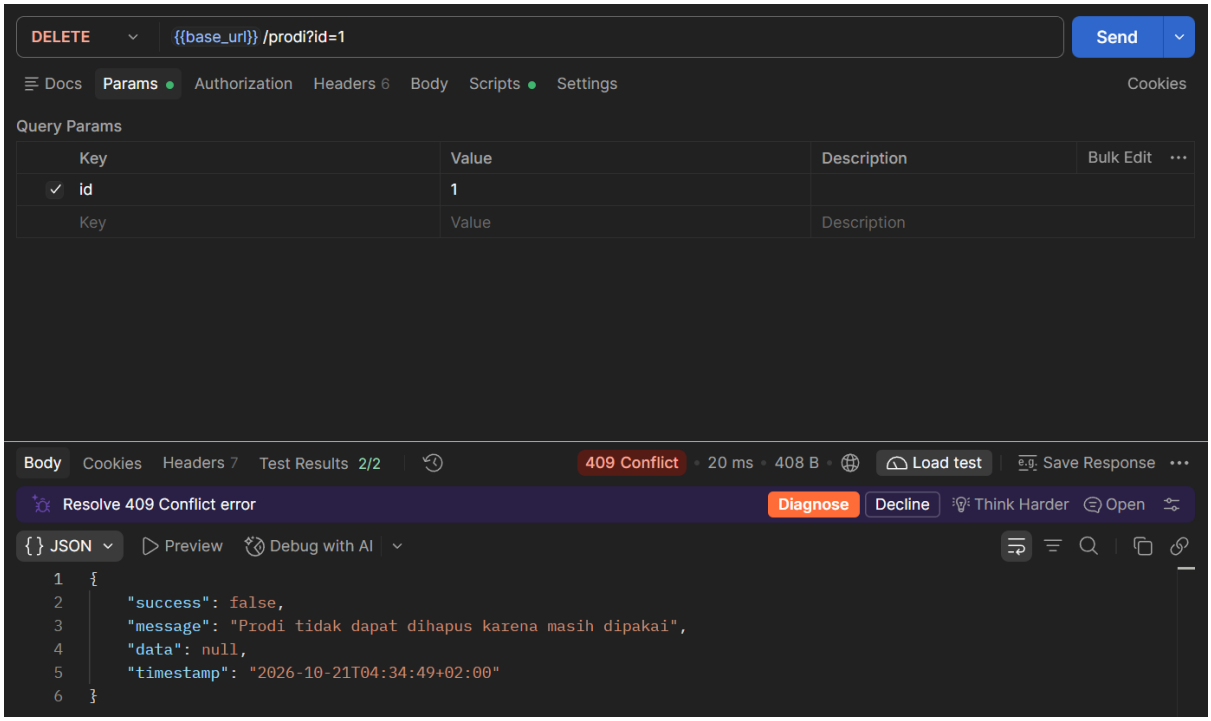
No	Request	Status	Hasil yang diharapkan
10	GET /buku (endpoint tidak ada)	404	Endpoint tidak ditemukan
11	PATCH /mahasiswa	405	Method tidak diizinkan

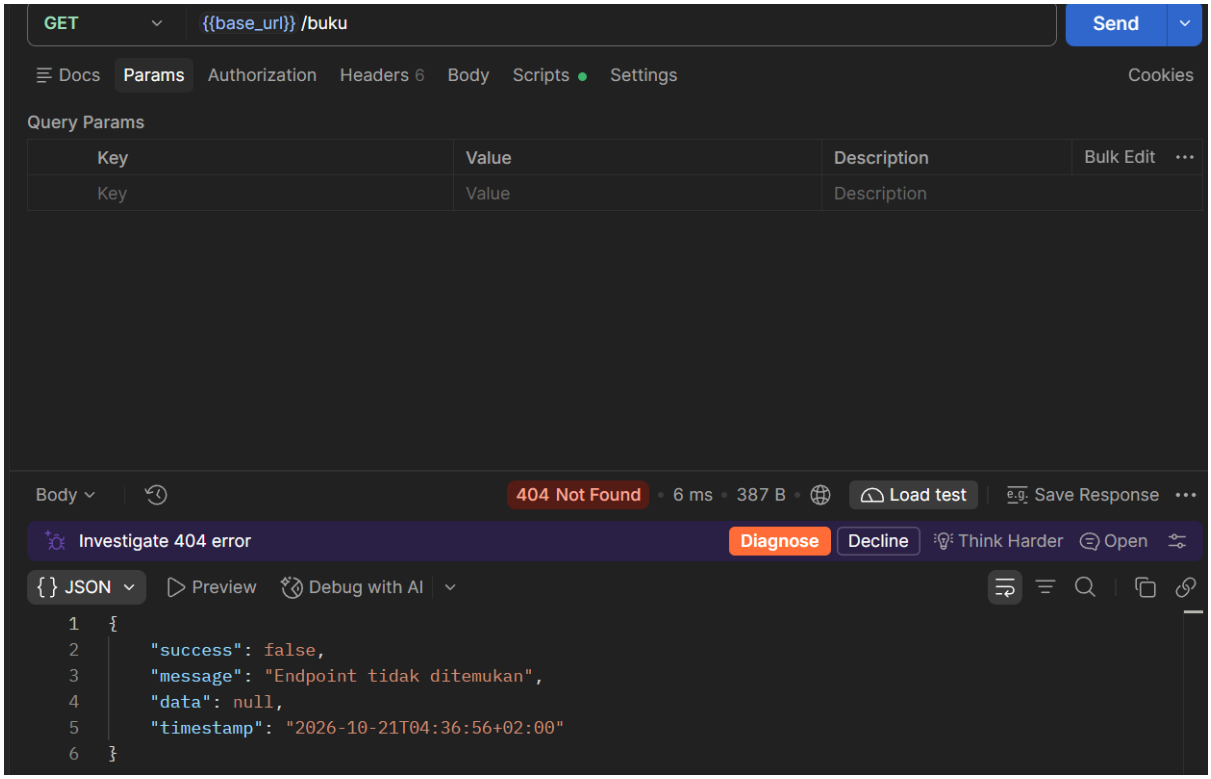


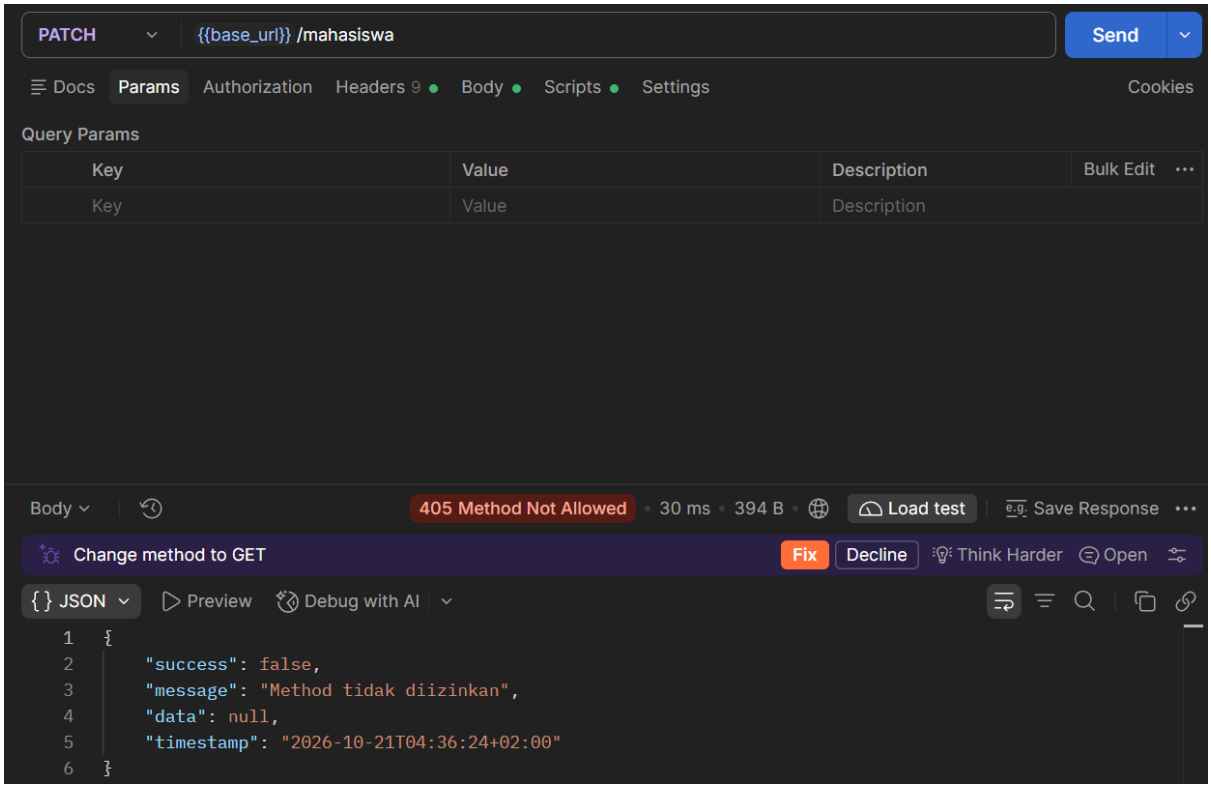



POST
{{base_url}} /mahasiswa
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
Cookies
Headers 6
Test Results 3/3
400 Bad Request
12 ms
408 B
Load test
Save Response
Add missing nim field
Fix
Decline
Think Harder
Open
JSON
Preview
Debug with AI
{
"success": false,
"message": "Data belum lengkap",
"data": null,
"errors": {
"nim": "Field nim wajib diisi"
},
"timestamp": "2026-10-21T04:33:04+02:00"
}


POST
{{base_url}} /mahasiswa
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
422 Unprocessable Entity (WebDAV) (RFC 4918)
31 ms
472 B
Load test
Save Response
Fix the NIM format
Fix
Decline
Think Harder
Open
JSON
Preview
Debug with AI
{
"success": false,
"message": "Validasi gagal",
"data": null,
"errors": {
"nim": "Format NIM tidak valid, harus 7 digit angka"
},
"timestamp": "2026-10-21T04:33:50+02:00"
}


DELETE
{{base_url}} /prodi?id=1
Send
Docs
Params
Authorization
Headers 6
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
id
1
Key
Value
Description
Body
Cookies
Headers 7
Test Results 2/2
409 Conflict
20 ms
408 B
Load test
Save Response
Resolve 409 Conflict error
Diagnose
Decline
Think Harder
Open
JSON
Preview
Debug with AI
{
"success": false,
"message": "Prodi tidak dapat dihapus karena masih dipakai",
"data": null,
"timestamp": "2026-10-21T04:34:49+02:00"
}


GET
{{base_url}} /buku
Send
Docs
Params
Authorization
Headers 6
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
404 Not Found
6 ms
387 B
Load test
Save Response
Investigate 404 error
Diagnose
Decline
Think Harder
Open
JSON
Preview
Debug with AI
{
"success": false,
"message": "Endpoint tidak ditemukan",
"data": null,
"timestamp": "2026-10-21T04:36:56+02:00"
}



Contoh bentuk response (berdasarkan data awal acara_7). Seluruhnya memiliki field success, message, data, dan timestamp:

```
{
    "success": true,
    "message": "Data berhasil ditambahkan",
    "data": {
        "id": "5"
    },
    "timestamp": "2026-10-21T10:30:00+07:00"
}
```

```
{
    "success": false,
    "message": "Data tidak ditemukan",
    "data": null,
    "timestamp": "2026-10-21T10:30:00+07:00"
}
```

```
{
    "success": false,
    "message": "Data belum lengkap",
    "data": null,
    "errors": {
        "nim": "Field nim wajib diisi"
    },
    "timestamp": "2026-10-21T10:30:00+07:00"
}
```

```
{
    "success": false,
    "message": "Validasi gagal",
    "data": null,
    "errors": {
        "nim": "Format NIM tidak valid, harus 7 digit angka"
    },
    "timestamp": "2026-10-21T10:30:00+07:00"
}
```

```
{
    "success": false,
    "message": "NIM sudah terdaftar",
    "data": null,
    "timestamp": "2026-10-21T10:30:00+07:00"
}
```

```
{
    "success": false,
    "message": "Method tidak diizinkan",
    "data": null,
    "timestamp": "2026-10-21T10:30:00+07:00"
}
```

7. Tugas Praktik Mandiri

Tugas 1: field errors. Method Response::json() diberi parameter kelima, $errors (array). Jika diisi, response memuat field errors berisi pesan kesalahan per field, seperti pada contoh response 400 dan 422 di atas. Jika tidak diisi, field errors tidak muncul.

Tugas 2: pretty print hanya mode development. Konstanta APP_ENV dibuat pada berkas konfigurasi baru app/config/app.php dan dimuat oleh public/index.php. Response::json() menambahkan JSON_PRETTY_PRINT hanya jika APP_ENV bernilai development. Bila diubah ke production, JSON menjadi satu baris (lebih kecil untuk dikirim lewat jaringan).

```
// Konfigurasi aplikasi.

// Mode aplikasi: 'development' (pengembangan) atau 'production'.
// JSON dibuat rapi (pretty print) hanya pada mode development.
define('APP_ENV', 'development');

// Panjang NIM yang dianggap valid. Data pada database acara_7 memakai
// NIM 7 digit (contoh: 2401001).
define('NIM_LENGTH', 7);
```

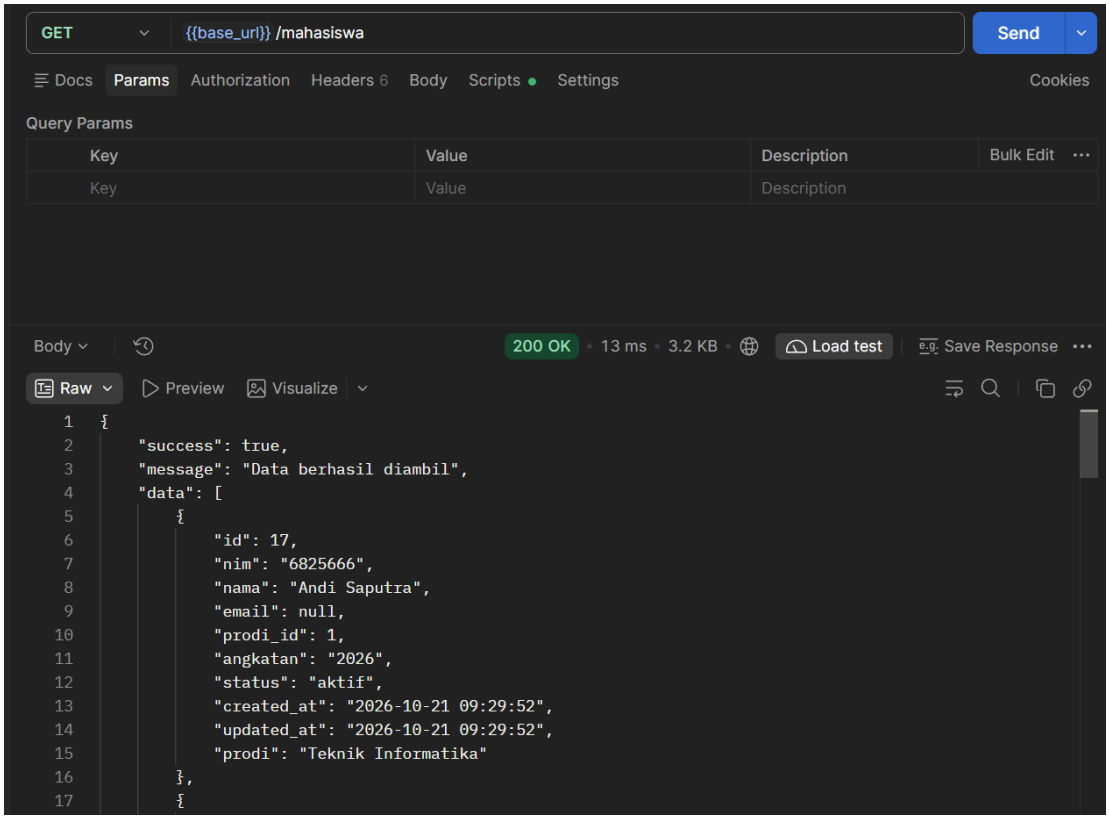
app/config/app.php

Contoh response pada mode production (satu baris):

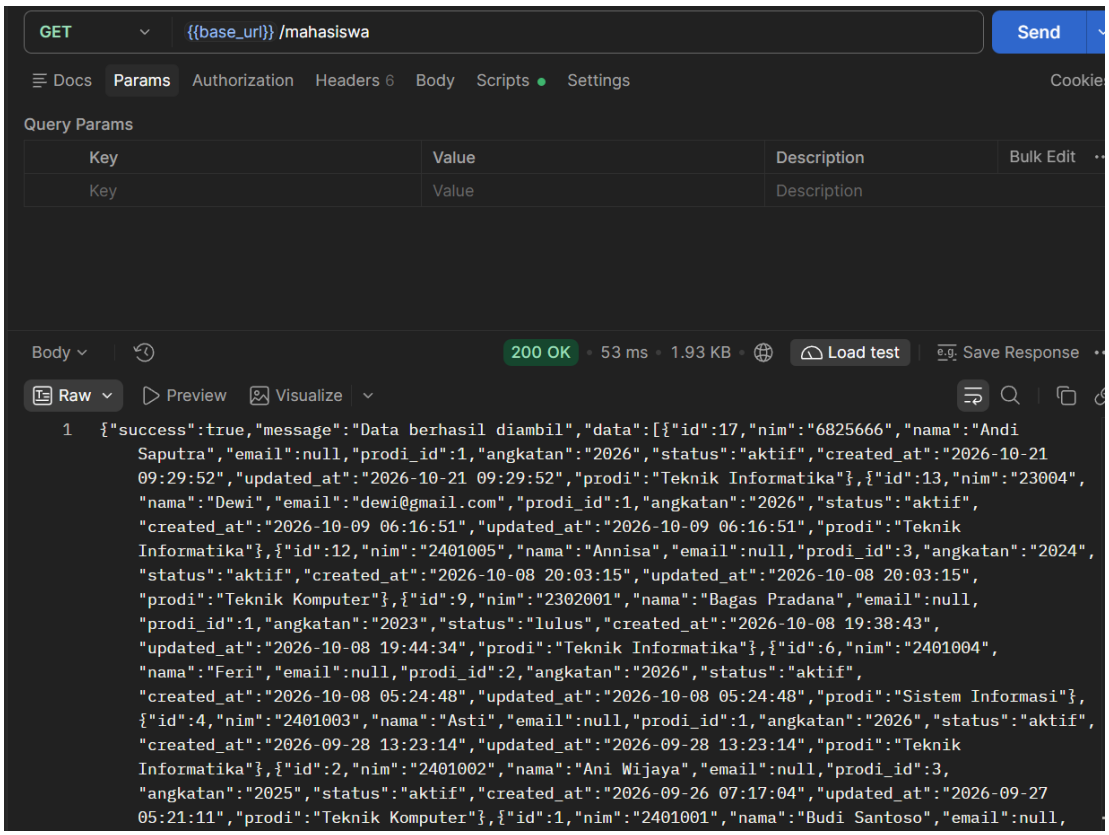
```
{"success":true,"message":"Data berhasil dihapus","data":null,"timestamp":"2026-10-
21T10:30:00+07:00"}
```

Response pada mode production

Pengujian mode development



Pengujian mode production



8. Pembahasan

- Dengan class Response, format JSON hanya ditulis di satu tempat. Jika format ingin diubah (misalnya menambah field), cukup mengubah satu file.
- Kode status yang tepat membantu client mengambil keputusan: 400 berarti data belum lengkap, 422 berarti formatnya salah, 404 berarti tidak ditemukan, dan 409 berarti bentrok dengan data lain.
- Pesan error database tidak dikirim ke client. Client hanya menerima pesan umum dengan kode 500.
- Pada API yang lebih besar dapat dipakai standar RFC 9457 (Problem Details). Praktikum ini memakai format envelope yang lebih sederhana.

KESIMPULAN

1. Class Response membuat seluruh response API memiliki format JSON yang seragam: success, message, data, dan timestamp.
2. Controller dan Router menjadi lebih ringkas karena memakai Response::json().
3. HTTP status code 200, 201, 400, 404, 405, 409, 422, dan 500 dipakai sesuai kondisi, dan seluruh endpoint berhasil diuji ulang termasuk skenario gagal.
4. Tugas mandiri berhasil dikerjakan: field errors untuk kesalahan validasi per field dan pretty print yang hanya aktif pada mode development.

DAFTAR PUSTAKA

Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Politeknik Negeri Jember.